

Seven ideas, and how to **prove** each one works

A validation protocol and a jury defence for every build candidate — plus the public German tax data that makes the proof possible. Built on a corpus already downloaded to this machine.

BUILD WINDOW	DEMO	TEAM	PRIZE
135 minutes 19:00 → 21:15	2–3 min Awards 21:40	3 minimum Pods of 2–3	€500 / €300 / €100 per person

The thesis of this document. Every team in that room will demo something that looks right. Taxfix's own job ads say excellence is *"knowing when the AI output is wrong even when it looks right."* So the differentiator is not the build — it's the **oracle**: an independent source of truth you can check yourself against, live, on stage.

For four of the seven ideas below, a genuine oracle exists in German public data and is already on this laptop. Those four are the strong candidates. The other three are validated differently, and this document says so plainly rather than pretending otherwise.

German tax is unusually well served by open data, because the state publishes the rules as machine-readable artefacts — including its own reference implementation of the tax arithmetic. Status below is as verified on this machine, 16 Sep 2026.

SOURCE	STATUS	WHAT IT ACTUALLY IS	VALIDATES
Gesetze im Internet <i>gesetze-im-internet.de</i>	ON DISK	Full statutory text as XML, free to reuse, with a published DTD and a daily-updated TOC. Downloaded: EStG (1.5 MB), UStG , AO , StBerG . Every paragraph addressable.	Citations, grounding
BMF Muster <i>Lohnsteuerbescheinigung 2026</i>	ON DISK	The official 4-page template for the printed wage tax statement, published 29 Aug 2025. This is <i>the</i> document Taxfix onboarding starts by photographing — field numbering and layout are prescribed here.	Extraction layout
KoSIT XRechnung test suite <i>projekte.kosit.org</i>	ON DISK	90 XML test instances of the German e-invoice standard, the official conformance corpus used by the state's own validator.	E-invoice parsing
Mustangproject fixtures <i>ZUGFeRD / Factur-X</i>	ON DISK	158 PDF+XML fixtures. Critically: these are hybrid invoices — a human-readable PDF with the structured truth embedded inside it . The answer key ships in the file.	Receipt OCR ground truth
BMF Lohn- & Einkommensteuerrechner <i>bmf-steuerrechner.de</i>	EMAIL ONLY	The government's own calculator, with an external interface the BMF publishes so third parties can machine-check against the official one. Endpoint verified live and returning structured XML — but it needs a <i>Zugriffscode</i> and there is no self-service signup: the only route is emailing Steuerrechner@bmf.bund.de . Not available for tomorrow. Use the PAP below instead.	Tax arithmetic, to the cent
BMF Programmablaufplan 2026 <i>PAP, XML pseudocode</i>	ON DISK	This is your oracle . The official wage-tax algorithm as XML pseudocode, explicitly published for transliteration into Java or C — because the BMF knows the graphical PAP is ambiguous about decimal handling. No gate, no signup: downloaded, 68 KB, 23 methods, 41 execution steps , all the real variables (STKL, RE4, KVZ, PVS...). Both 2025 and 2026 on disk.	Wage tax, offline
Destatis Lohn- u. Einkommensteuerstatistik	ON DISK	Official distributions of income, deductions and refunds, plus a REST/JSON GENESIS API. Use for plausible synthetic personas — incomes drawn from the real distribution, not invented.	Realistic test cases
ELSTER / ERic SDK	NOT TONIGHT	The C library for actually transmitting returns. Free, but requires developer registration and a manufacturer ID. Out of scope for 135 minutes — reference it as the integration path, don't attempt it.	—
SROIE / CORD <i>receipt OCR benchmarks</i>	OPTIONAL	Public annotated receipt datasets (~1k images each), but retail receipts without VAT structure. The ZUGFeRD fixtures above are strictly better here .	Fallback OCR

What this corpus does and does not give you

It gives you: the law as structured data, the government's own arithmetic as runnable pseudocode, the exact document format Taxfix ingests, and invoices where the correct parse is embedded in the file. An unusually strong position for a hackathon.

It does not give you: real *Steuerbescheide*, *Finanzamt* correspondence, or user returns — personal tax data, correctly not public. For ideas needing them, **synthesise the test set and say out loud that you did**. "We generated 40 notices with known injected errors" beats "we tried one example," and a stated limitation beats a quiet one.

THE ONE MOVE THAT PAYS FOR ITSELF

Implement the PAP 2026 XML. It is the Bundesfinanzministerium's own wage-tax algorithm, on this laptop, with no gatekeeper — and it converts idea 07 from "we think this is right" into "this is the federal calculation, transliterated, and here is the diff running live." The BMF publishes it in exactly this form *because* they want third parties to get the arithmetic right. No other team will have that. One warning the BMF gives on the page itself: **use BigDecimal, not floats** — rounding drift is the one thing that will silently break a cent-exact claim on stage.

The same five-part skeleton applies to all seven ideas. Fill it in before you write code — it takes four minutes and it determines what your demo can claim.

ORACLE	The independent source of truth. Ranked best to worst: (1) a government reference implementation, (2) ground truth embedded in the test file itself, (3) a test set you constructed, so you know the answer by construction, (4) a human judgment you recorded beforehand. Never: the same model grading itself.
TEST SET	Small and adversarial beats large and easy. 15–40 cases is plenty. Include at least three cases designed to break your own system, and keep them in the demo.
METRIC	One headline number a judge can hold. Prefer asymmetric metrics that reflect real cost: in tax, a missed deduction is annoying, a wrong figure filed with the <i>Finanzamt</i> is a legal problem. Measure those separately, never as one accuracy score.
PASS BAR	Declared <i>before</i> you run it. "≥95% extraction F1 and zero silent errors" is a bar. "It works well" is not.
FAILURE MODE	Name what wrong looks like, and show the system catching it on stage. This is the single highest-value 20 seconds of your demo.

The asymmetry that matters in this domain

Tax software has two failure classes and they are not equally bad:

FAILURE	COST	CORRECT SYSTEM BEHAVIOUR
Over-escalation flags something a human didn't need to see	Low wasted minutes	Acceptable. Tune towards this.
Silent error auto-decides, and is wrong, and nobody is told	Severe filed, legal, liability	Must be zero. Every auto-decision needs a confidence floor and an audit line.

Say this out loud during your demo. It shows you understand that Taxfix is a regulated financial product and not a chatbot — which is precisely the argument the company is making about itself right now.

Scoring the seven, before you pick

#	IDEA	ORACLE QUALITY	BUILDABLE IN 135 MIN	DEMO LEGIBILITY	VERDICT
01	Expert co-pilot	Constructed	Yes	Medium	Take if brief is internal tools
02	Bescheid-Check	Constructed, exact	Yes	Very high	Default pick
03	Finanzamt agent	Constructed	Yes	High	Strong, close to 02
04	Self-employed books	Embedded in file	Tight	Medium	Best oracle of all
05	Embedded SDK	None — integration	Yes	Medium	Strategy swing, risky
06	Verification harness	ESTG XML	Yes	High	On-thesis, needs prep
07	Günstigerprüfung	Federal ministry	Yes	Medium	Unbeatable proof

The tax expert's co-pilot

Drop in a client's document bundle. Out comes a pre-filled draft return, plus a ranked queue of exactly the items a licensed human must decide — each with the reason, the euro at stake, and the source region highlighted.

THE PAIN IT HITS

Expert Service is priced at 20% of refund, min €99.99 — their highest-revenue line and their worst margin, because it needs a licensed *Steuerberater*. Germany cannot supply them: **72% of tax firms report unfillable vacancies**, over half of practising advisors are 50+, and 2026 is the first year more Germans retire than join the workforce.

They cannot hire their way out. Only productivity works.

DATA YOU HAVE

BMF *Lohnsteuerbescheinigung* Muster 2026 (prescribed field numbering), ZUGFeRD fixtures as expense documents, Destatis distributions for realistic incomes.

Construct 12 cases across three difficulty tiers: clean, ambiguous, and one containing a deliberate contradiction between two documents.

ORACLE	Constructed. You build each case from known inputs, so the correct extraction and the correct escalation decision are known by construction. Record the expected answers in a <code>cases.json</code> before you build the system.
TEST SET	12 cases × ~20 fields. Tier A clean (5), Tier B ambiguous (5), Tier C contradictory (2).
METRIC	Two numbers, reported separately. (1) Field extraction F1 against the known values. (2) Escalation confusion matrix — of items auto-decided, how many were wrong (<i>silent error rate</i>); of items escalated, how many genuinely needed a human (<i>escalation precision</i>).
PASS BAR	Extraction F1 ≥ 0.95 • silent error rate = 0 • both Tier C contradictions escalated. Over-escalation up to ~20% is an explicit pass, not a failure.
FAILURE MODE	The system confidently reads a figure from the wrong line of the wage statement and files it without flagging. Demo this: run Tier C live and show the contradiction caught and routed to a human.

Defending it to the jury

Open: *"Your highest-revenue product needs a licensed human, and Germany has run out of them — 72% of tax firms can't fill roles and half the profession is over 50. You can't hire out of this. So we didn't build something that replaces the expert. We built the thing that decides what the expert never has to look at."*

The number: expert-touch items per case, before and after. "Twenty decisions become four, and the four are ranked by euros at stake."

“Isn't this just OCR? We already do document scanning.”

→ The extraction isn't the product — the *triage* is. We're not claiming to read documents better than you. We're claiming to know which four of twenty items a human must see, and we measured that as a confusion matrix rather than an accuracy number.

“What happens when it's wrong?”

→ We separate the two failure classes. Over-escalation costs minutes; a silent wrong auto-decision is a filing liability. Our pass bar allows the first and requires zero of the second. Here's the case where it catches itself.

“Your test cases are synthetic.”

→ Yes, deliberately — real returns are personal data we shouldn't have. Synthetic means we know the right answer by construction, which is exactly what lets us report a silent-error rate at all. With your real corpus this protocol runs unchanged.

Bescheid-Check

Photograph the assessment notice. The tool diffs it line by line against what was filed, flags every deviation in plain German and English with the euro impact, and where a deviation looks wrong, drafts the *Einspruch* with the deadline counting down.

THE PAIN IT HITS

Six to twelve weeks after filing, a dense multi-page *Steuerbescheid* arrives. The objection window under §355 AO is **one month**. Finanztip's 2026 review names this as a Taxfix weakness in exactly these terms: checking the assessment is **"only possible in a limited way, or expensively."**

It is also a **year-round touchpoint** on a once-a-year product — which is the company's stated strategic problem.

DATA YOU HAVE

You generate both sides, which is what makes the oracle exact. Take a filed-return object, render a synthetic *Bescheid* from it, then **inject N known deviations** of known euro value. **ESTG XML** supplies the paragraph citations for each deviation type.

ORACLE	Exact and constructed. You injected the deviations, so you know precisely which ones exist, where, and what each is worth. This is the cleanest oracle of the three "constructed" ideas because it has zero interpretive ambiguity.
TEST SET	40 synthetic notices . 10 clean (no deviation — tests false positives), 25 with 1–3 injected deviations spanning value bands, 5 adversarial: a deviation that is <i>correct in law</i> (the <i>Finanzamt</i> was right) and must not be flagged as an error.
METRIC	Recall on injected deviations , banded by euro value. False positives per clean notice . Euro-impact accuracy (absolute error on the stated amount). Deadline arithmetic — exact date, no tolerance.
PASS BAR	100% recall on deviations $\geq \text{€}50$ · < 1 false positive per notice · euro impact exact · deadline exact on all 40 · all 5 adversarial cases correctly not flagged.
FAILURE MODE	Crying wolf. A tool that flags a lawful adjustment as an error teaches users to file pointless objections and destroys trust in one step. Demo the adversarial case: show it recognising that the tax office was right.

Defending it to the jury

- Open:** "Finanztip's 2026 review says checking your *Bescheid* with Taxfix is only possible in a limited way, or expensively. That's the moment your user is most anxious and you're thinnest. It's also the one moment that brings them back in November. Here it is, fixed."
- The number:** "Across 40 notices we caught every deviation above €50, with fewer than one false positive per notice — and we correctly stayed quiet on five where the tax office was right."
- “Users could just read the notice themselves.”**
- They don't, which is why this is a known problem. And the clock is the product as much as the diff — §355 AO gives one month, and a user who reads it in week five has no remedy left.
- “How do you know your deviations are the ones that matter?”**
- We banded by euro value and report recall per band rather than one blended number, because a €4 rounding difference and a €400 rejected deduction are not the same event. Above €50 we're at 100%.
- “Aren't you going to generate a lot of pointless objections?”**
- That's the failure mode we optimised against first. Five of our forty cases are deviations where the *Finanzamt* was legally correct, and the pass bar requires we flag none of them. Watch — this is that case.
- “This is a feature, not a business.”**
- It's a retention mechanism. Your structural problem is a €45 transaction once a year with no reason to open the app in between. This puts a real reason in week eight, and it's the moment the user is most receptive to an upsell into Expert Service.

The Finanzamt correspondence agent

Upload the letter from the tax office. It classifies what is actually being asked, explains it in plain language, lists precisely which documents satisfy the request, and drafts the reply with the statutory deadline tracked.

THE PAIN IT HITS

The single most repeated complaint in Taxfix reviews: the app is excellent right up until the *Finanzamt* writes back, and then users feel abandoned mid-process. Reviews cite **eight messages before a reply**. Support demand also spikes hard into the 31 July deadline — the same window where staff are most stretched.

DATA YOU HAVE

AO XML for the procedural deadlines, EStG XML for the substantive grounds. Letter types are standardised and publicly documented in tax guidance — write 15 from templates, covering the common request classes.

ORACLE	Constructed, with a labelled taxonomy. Define ~8 request classes (proof of expenses, missing attachment, clarification of income, notice of intended deviation, etc.), write 15 letters against them, label each with its class, its required-document list, and its deadline.
TEST SET	15 letters. Include two where the letter asks for more than one thing — multi-intent is where these systems quietly fail.
METRIC	Intent classification accuracy. Document-list completeness — treat any under-listing as a failure, since a user who replies with 3 of 4 documents triggers another round trip and another month. Deadline extraction exactness.
PASS BAR	Zero missed deadlines · zero under-listed document sets · both multi-intent letters fully decomposed. Over-listing documents is acceptable; under-listing is not.
FAILURE MODE	The letter asks two things and the agent answers one. Demo this: feed the multi-intent letter and show both requests surfaced separately.

Defending it to the jury

Open: *"We read your reviews. The loudest recurring complaint is that support ends the moment the tax office writes back. That's also your worst support cost, in your worst week of the year. This is that complaint, answered."*

The number: "Deflected round trips. Every under-listed document set costs the user a month and costs you a support thread — we made under-listing a hard failure in our pass bar, and we have zero."

"This is a support automation, not a product."

→ It's both, and the support saving is the part you can bank this year. But it also fixes the reason users say they'd not recommend you — which is a CAC argument, not a support argument.

"Tax office letters are too varied to classify."

→ They're more standardised than they look — we built against eight classes and the two we deliberately made multi-intent are the interesting ones. Here's the decomposition.

"What if it gives wrong advice? You're not tax advisors."

→ Correct, and that's designed in. It never asserts a tax position — it classifies the request, lists documents, and tracks the deadline. Anything touching a substantive position routes to your licensed staff. That's the *Steuerberatungsgesetz* line and we stay on the right side of it.

Always-on bookkeeping for the self-employed

Receipts arrive by email forward, photo or bank feed. The agent categorises, posts, keeps a running *EÜR*, drafts the monthly *UStVA* — and warns before the §19 UStG thresholds are crossed, because crossing one changes your legal obligations overnight.

THE PAIN IT HITS

The €19.99/month self-employed product launched March 2026 is their only recurring revenue and the only product team they're actively hiring engineers for. Market: ~3.7m self-employed Germans. They are the newest entrant against Accountable, sevDesk, Lexware and Norman — and their app currently **stops working entirely above the Kleinunternehmer threshold**.

DATA YOU HAVE

This is the strongest oracle on the list. ZUGFeRD/Factur-X fixtures are hybrid files: a human-readable PDF *with the structured ground truth embedded inside the same document*. Plus **90 KoSIT XRechnung conformance instances**. The answer key ships in the file.

ORACLE	Embedded. Parse the PDF as a human would — visually — then compare against the XML payload inside that same PDF. No labelling, no construction, no judgment call. Machine-checkable at line-item granularity across ~150 fixtures.
TEST SET	~30 ZUGFeRD PDFs for extraction, plus the 90 KoSIT XML instances for structural conformance. Include the <code>corrupt-factur-x-waytoosmall.pdf</code> fixture — malformed input is a real production case.
METRIC	Field-level accuracy vs embedded XML. VAT reconciliation to the cent — net + VAT must equal gross, and the VAT rate must match the line item. Threshold alarm timing — fires <i>before</i> cumulative turnover crosses €25,000, not after.
PASS BAR	Cent-exact VAT reconciliation on 100% of fixtures · threshold alarm fires before crossing · malformed input fails loudly rather than silently posting a wrong entry.
FAILURE MODE	Silently posting a wrong VAT rate. This propagates into a filed <i>UStVA</i> — a legal document. Demo the corrupt fixture and show it refusing to post.

Defending it to the jury

Open: "Your €19.99 subscription is the only recurring revenue you have, and it's the only place you're hiring product engineers. So we built for it — and we validated against the government's own e-invoice conformance corpus, where the correct answer is embedded inside the file. Cent-exact, across a hundred and fifty invoices."

The number: "Cent-exact VAT reconciliation on 100% of fixtures. Not sampled — all of them."

“Accountable, sevDesk and Lexware already do this.”

→ They do bookkeeping. None of them owns the filing relationship and the licensed advisor on the other side of it. Your wedge isn't the ledger, it's that you can take the ledger all the way through ELSTER and stand behind it. Also — the threshold alarm is a tax-advice feature, not a bookkeeping feature, and it's the moment a freelancer needs you most.

“Why does the threshold warning matter so much?”

→ Because crossing €25,000 changes your legal obligations overnight, and the current app just stops being able to help you at exactly that point. That's your most valuable customer becoming someone else's on the day they get successful.

“E-invoicing isn't mandatory for small businesses.”

→ Issuing isn't — *Kleinunternehmer* are exempt under §34a UStDV. But **receiving structured e-invoices has been mandatory for every German business since 1 January 2025, with no exemption**. Every one of your users is already receiving files they can't read. That's the wedge.

Taxfix Embedded — tax as an API

A drop-in component a neobank, payroll provider or HR platform embeds so their users file without ever leaving that app — with Taxfix taking the fee and owning the regulated last mile.

THE PAIN IT HITS

Every Taxfix partnership today is a **discount link**. N26 users get 30–50% off; Qonto and employer deals work the same way. Not one of them is an API. A partner cannot build tax *into* their product — they can only hand the user away with a coupon, which compresses an already thin ~€45 AOV and leaks the relationship.

BE HONEST ABOUT THIS ONE

There is no accuracy oracle here, because correctness isn't the question — adoptability is. Don't fake a metric. Validate it as an integration problem, which has its own perfectly respectable measurements.

ORACLE	None, and say so. Substitute: a cold integrator . Recruit one person at the event who has not seen your code, hand them the snippet, and time them.
TEST SET	Three host environments with deliberately conflicting styling — e.g. a Tailwind app, a Bootstrap app, and a plain-CSS app with an aggressive global reset. Style bleed in either direction is the classic embed failure.
METRIC	Lines of host code to first working embed. Time to integrate , measured on the cold integrator. Style isolation — pass/fail per host. Data boundary — can the host read the user's tax data? It must not.
PASS BAR	< 20 lines of host code · < 10 minutes for someone who has never seen it · zero style bleed across all three hosts · host provably cannot read tax data.
FAILURE MODE	It works in your demo host and nowhere else. Demo all three hosts , switching live. That switch is the whole proof.

Defending it to the jury

Open: *"Every partnership you have is a discount link. N26 users get 50% off and then leave your partner's app. We think the asset isn't the app — it's that you're the one who can legally file. So we made that embeddable, and here it is running inside three different host apps."*

The number: "Eleven lines of host code. Ten minutes for someone who'd never seen it. Three hosts, no style bleed."

“Our partners don't want to build this.”

→ They don't want to build tax — they want the retention. A neobank that hands its user to a different app in filing season loses the most engaged session of the year. This lets them keep it and lets you keep the fee instead of discounting it away.

“This is a two-year platform project, not a hackathon demo.”

→ Agreed — which is why we scoped it to the thing that's actually in doubt: whether it embeds cleanly in a host you don't control. That's testable tonight and it's the part that kills these projects. The rest is your existing stack.

“What about liability and data protection?”

→ That's the design constraint, not an afterthought. The host never touches tax data — it's isolated, and the host app can't read it. We tested that as a pass/fail, because the moment a partner can read a user's income you have a very different regulatory conversation.

The answer to "why not just use ChatGPT?"

A verification harness. The same real German tax questions run through a general model and through a Taxfix-grounded pipeline that cites the actual paragraph — with a live scoreboard showing where the general model is confidently, checkably wrong.

THE PAIN IT HITS

OpenAI reported tax-related ChatGPT queries up 300% in Q1 2026. German trade press is writing about Taxfix in explicitly existential terms. This is the question their CEO is being asked in every board meeting, and it is *literally* their stated hiring criterion: "knowing when the AI output is wrong even when it looks right."

DATA YOU HAVE

EStG, UStG, AO and StBerG as XML, on this laptop. Every paragraph addressable. Which means every citation the system emits is **resolvable live, on stage** — and so is every citation the general model invents.

ORACLE	The statute itself. Two-layer: (a) citation validity — does the cited paragraph exist in the XML, and does its text support the claim? This is mechanically checkable. (b) answer correctness — graded against the paragraph, by you, recorded beforehand.
TEST SET	25 questions written before you build anything. Mix: 10 that a general model handles fine (be fair — this makes the comparison credible), 10 German-specific and current-year sensitive, 5 where the correct answer is "this depends on facts you haven't given me" — the trap a general model falls into by answering anyway.
METRIC	Citation resolution rate — % of cited paragraphs that exist and say what's claimed. Hallucinated citation count. Appropriate-refusal rate on the 5 underspecified questions.
PASS BAR	100% of your pipeline's citations resolve in the EStG/UStG XML · your system declines all 5 underspecified questions · you report the general model's numbers honestly, including where it beat you.
FAILURE MODE	Building a rigged comparison. A jury of engineers will smell it instantly and you lose the room. Include the cases where the general model was right — it makes the cases where it wasn't land far harder.

Defending it to the jury

Open: "Your own job ad says excellence is knowing when the AI output is wrong even when it looks right. We took that literally. Here are twenty-five tax questions, two systems, and every citation checked live against the statute — including the ten where the general model was perfectly fine."

The number: "Every citation our pipeline emits resolves in the EStG. The general model invented [N]. And on five questions where the honest answer is 'I need more facts,' it answered anyway — all five times."

“You've rigged the questions.”

→ Ten of the twenty-five are ones we expected it to get right, and it did — we report those. The interesting set is the five underspecified ones, where confidently answering is the failure. That's not a question-selection artefact, it's a behavioural difference.

“Models will get better. This advantage evaporates.”

→ The grounding isn't the moat and we're not claiming it is. The moat is that you can file, you carry the liability, and you have ten million returns with known *Finanzamt* outcomes — what was claimed, what was accepted, what was rejected. No foundation model has that. This harness is how you'd measure a model trained on it.

“This is a benchmark, not a product.”

→ It's a product surface too — it's the trust panel you put in the app when a user asks why they should pay you. But you're right that its first customer is internal: it's the regression suite for every AI feature you ship this year.

Günstigerprüfung for couples

Two incomes in. Both assessment paths computed — *Zusammenveranlagung* versus *Einzelveranlagung* — and the euro difference out. The feature Taxfix is publicly criticised for not having.

THE PAIN IT HITS

Finanztip's 2026 review states it plainly: with Taxfix, a **married couple cannot check whether joint or separate assessment is better for them**. It's a named, published gap in the test that most shapes German purchase decisions — where Taxfix placed sixth at the highest price in the field.

WHY THIS ONE IS SPECIAL

You can run the German finance ministry's own arithmetic. The BMF publishes its wage-tax algorithm as XML pseudocode precisely so third parties can reproduce the official calculation exactly. It is on this laptop. No other idea here has an oracle with that authority.

ORACLE	BMF Programmablaufplan 2026 — the federal reference algorithm as XML pseudocode, on disk, ungated. Transliterate it and you are running the ministry's own arithmetic. <i>The live BMF API would also work but needs a Zugriffscode obtainable only by emailing the BMF — not available on this timescale, so don't plan around it.</i>
TEST SET	A grid of ~200 income pairs sampled across the Destatis income distribution, plus edge cases: zero-income spouse, both at the same income (where splitting gains nothing), one spouse just over a progression threshold, and a pair straddling the top rate.
METRIC	Absolute cent deviation from the BMF figure, per case. Sign correctness of the recommendation — did you name the right winner? Magnitude accuracy of the stated saving.
PASS BAR	Cent-exact against BMF on all 200 tariff computations · recommendation sign correct on 100% · zero cases where you recommend the worse option.
SCOPE HONESTY	State this in the demo. The BMF endpoint computes the tariff under §32a EStG and wage tax per the PAP — not a full assessment, which the <i>Finanzamt</i> does on individual circumstances. So the <i>tariff engine</i> is validated against the ministry, the <i>assessment-choice logic</i> against constructed cases. Saying so is a credibility gain, not a caveat.

Defending it to the jury

Open: "Finanztip's 2026 test says Taxfix can't tell a married couple whether joint or separate filing is better. That's a published gap in the review that moves German purchase decisions, and you're the most expensive app in that table. Here it is — and every number on this screen is cent-exact against the Bundesfinanzministerium's own calculator."

The number: "Two hundred income pairs. Zero cents of deviation from the federal calculator. Zero cases where we'd have recommended the worse option."

“That's a small feature.”

→ It is — and it's small, named, published, and currently missing, in the one review your German customers actually read while deciding whether to pay €49.99 or take the free one. Small and shipped beats large and hypothetical tonight.

“How do we know your maths is right?”

→ You don't have to take our word for it — it isn't our maths. We transliterated the BMF's own published algorithm, which they publish in exactly this form so third parties reproduce the official result. Two hundred cases, here's the diff, live.

“The full assessment is more complex than the tariff.”

→ Correct, and we're explicit about it: the ministry's endpoint validates the tariff under §32a, not the whole assessment. The assessment-choice logic is validated separately against constructed cases. We'd rather show you exactly where the federal oracle stops than blur the line.

“Why hasn't our team just built this?”

→ Because it's unglamorous and the splitting logic is fiddly. That's the point — fiddly, correctness-critical, and now provably right against the government's own numbers. A week of your roadmap, done.

Check-in is 18:00; the challenge drops at 19:00. That hour is the highest-leverage part of the evening, because Taxfix staff will be in the room and unguarded. Questions that make people talk are open, specific, and slightly uncomfortable.

To a Taxfix engineer or product person

- 1. "Where does an expert's time actually go on a case — what's the part you wish you could just delete?"
The single best question of the night. It hands you a build target from the person who feels the pain, and it's the one question nobody else will ask.
- 2. "What breaks between Taxfix, Steuerbot and the UK product — is it genuinely one platform yet?"
Three brands from two acquisitions. Integration debt is where the unglamorous, high-value problems live, and people are usually candid about it.
- 3. "When the state's pre-filled return goes live properly, which of your users stops needing you?"
Pointed, and shows you understand their year. Whatever they answer names the segment they're defending — build for that segment.
- 4. "What's the drop-off point in the funnel that annoys you most?"
The CPO came from N26 and a crypto exchange, owning onboarding and KYC. She was hired for the funnel. Whoever answers this is telling you her roadmap.
- 5. "What did the last internal AI hackathon produce that you actually shipped?"
They run these regularly. This tells you the real bar, and stops you pitching something they built in March.

To the organisers, in the first ten minutes

- "Is the challenge scored on the demo, the code, or the idea?" Changes where you spend minute 100 onwards.
- "Is there an API, a dataset, or a sandbox being provided?" If Taxfix hands out data, use it — building on their artefacts beats building on yours.
- "How strict is the 2–3 minutes?" Determines whether you get one demo beat or three.
- "Who is judging?" If the CTO is judging, lead with engineering rigour. If the CPO is, lead with the user moment.

Questions to have ready when the jury turns it back on you

Have a crisp answer prepared for each of these. They come up in almost every hackathon Q&A, and hesitating on them costs more than a missing feature.

THEY WILL ASK	YOUR SHAPE OF ANSWER
"How do you know it works?"	Name the oracle in one sentence, then the headline number, then offer to run the adversarial case live.
"What did you not build?"	Answer honestly and fast. "ELSTER submission — that needs a manufacturer ID and it's your existing integration anyway." Confident scoping reads as judgment.
"How long to production?"	Separate the model work from the compliance work. The compliance work is always longer and saying so proves you understand the domain.
"What's the business case?"	One of three currencies: expert minutes saved, support threads deflected, or a reason to open the app outside filing season. Pick one, don't list three.
"Who owns the liability when it's wrong?"	Your system never asserts a tax position without a human in the loop — and here's the escalation path. This is the answer that separates you from a chatbot demo.

Before you sleep

- ☐ **Open data/truth/Lohnsteuer2026-PAP.xml** and skim the METHOD blocks. This is the federal wage-tax algorithm and it is idea 07's entire proof. Highest-value 15 minutes available to you.
- ☐ **Do not plan around the live BMF API.** It needs a *Zugriffscore* with no self-service signup — email only, to Steuerrechner@bmf.bund.de. Worth sending for later; useless for tomorrow.
- ☐ **Unzip the ZUGFeRD fixtures** and open two of them — confirm you can read the embedded XML out of the PDF. If that pipeline works, idea 04 is live.
- ☐ **Open the Lohnsteuerbescheinigung Muster** and note the numbered fields. That numbering is prescribed, which means extraction can be anchored to it rather than guessed.
- ☐ **Pick your primary and your fallback** from the seven — before the brief drops, so the brief only tells you which to reach for. Suggested: **02 primary, 01 fallback if the brief is internal tooling.**
- ☐ **Write your 25 questions / 40 notices / 200 income pairs tonight** if you can. Test sets are the thing you will not have time to write tomorrow, and they're the thing that makes the demo a proof instead of a claim.
- ☐ **Check your laptop runs Chrome headless, has the corpus, and your Cursor is signed in.** Event credits are provided; don't spend build time on setup.

The clock

- 18:00** Check-in. **Lock your team of three now** — organiser matching costs you the first twenty minutes.
- 18:30** Welcome. Ask the organisers the four scoping questions from §D.
- 18:45** Cursor session. Note anything they demo that you can exploit.
- 19:00** Challenge drops. **Spend four minutes filling in the §B skeleton** — oracle, test set, metric, pass bar, failure mode — before anyone writes code. This is the discipline that wins.
- 19:10** Split: one owns the working demo path end to end, one owns data and validation, one owns narrative and the closing number. Nobody "does the backend."
- 20:30** **Feature freeze.** Whatever isn't working now isn't going in.
- 20:45** Stop building. Run the validation, get the real number, put it on a slide.
- 21:00** Rehearse the exact click sequence twice. **Record a fallback screen capture.** More demos die on a live API call than on a missing feature.
- 21:15** Demos. Open with their pain in their words. Show the failure being caught. Close on one number.
- 21:40** Awards.

THE THREE MINUTES, STRUCTURED

- 0:00–0:25 — The pain, quoted.** Their review, their Finanztip score, their job ad. You will be the only team that read them.
- 0:25–1:40 — The thing working.** Live, on a real German tax document. No slides in this stretch.
- 1:40–2:10 — The catch.** Feed it the adversarial case. Show it refusing, escalating, or catching itself. This is the twenty seconds that wins.
- 2:10–2:40 — The proof.** The oracle, the test set size, the one number.
- 2:40–3:00 — What's next, honestly.** Name what you didn't build and why. Stop talking.